



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Diseño y Programación de Bases de Datos	Unidad #: 3	Tema: Manejo de seguridades e introducción al PL /SQL
	Fecha:	Tutor:

Introducción:

Bienvenidos al material de lectura 1 de la Unidad #3 de la materia Diseño y Programación de Base de Datos .

PL/SQL es un lenguaje de programación procedimental diseñado por Oracle para optimizar la gestión de bases de datos mediante la integración de estructuras de control, manejo de excepciones y modularidad en la ejecución de sentencias SQL. A diferencia del uso interactivo de SQL o de simples scripts, PL/SQL permite estructurar el código en bloques organizados que mejoran el rendimiento, facilitan la reutilización y reducen errores en entornos de producción. Su compatibilidad con la Programación Orientada a Objetos (POO) y su capacidad para ejecutar órdenes de manipulación de datos lo convierten en una herramienta esencial para desarrolladores y administradores de bases de datos.

Identifica y resume los puntos clave del texto. Anota ideas principales, términos relevantes y cualquier concepto que consideres esencial:

Además, la ejecución de los programas en el servidor, conlleva un ahorro de recursos en los clientes y disminución del tráfico de red.

El uso del lenguaje PL/SQL es también imprescindible para construir disparadores de bases de datos que permiten implementar reglas complejas de negocio y auditoría en la base de datos.

3.1 Características de PL/SQL

Por todo ello, el conocimiento de este lenguaje es imprescindible para poder trabajar en el entorno Oracle, tanto para administradores de la base de datos como para desarrolladores de aplicaciones.

PL/SQL es un lenguaje procedimental diseñado por Oracle para trabajar con la base de datos. Está incluido en el servidor y en algunas herramientas de cliente. Soporta todos los comandos de consulta y manipulación de datos.

La unidad de trabajo es el **bloque**, constituido por un conjunto de declaraciones, instrucciones y mecanismos de gestión de errores y excepciones.

3.1.1 Bloques PL/SQL

El bloque es la estructura básica característica de todos los programas PL/SQL. Tiene tres zonas claramente definidas:

- Una zona de declaraciones donde se declaran objetos locales (variables, constantes, etcétera). Suele ir precedida por la cláusula DECLARE (o IS/AS en los procedimientos y funciones). Es opcional.
- Un conjunto de instrucciones precedido por la cláusula BEGIN.
- Una zona de tratamiento de excepciones precedido por la cláusula EXCEPTION. Esta zona, igual que la de declaraciones, es opcional.

El formato genérico del bloque es:

```
[ DECLARE  
<declaraciones> ]  
BEGIN  
<órdenes>  
[ EXCEPTION  
<gestión de excepciones> ]  
END;
```

La utilización de bloques supone una notable mejora de rendimiento ya que se envían los bloques completos al servidor para que sean procesados, en lugar de cada sentencia SQL. Así se ahorran muchas operaciones de E/S.

Los bloques PL/SQL se pueden anidar para conseguir distintas funcionalidades (por ejemplo, para evitar la propagación de excepciones) como veremos más adelante. A estos bloques se les llama **bloques anidados**.

```
DECLARE
...
BEGIN
...
DECLARE -- comienzo del bloque interior anidado
...
BEGIN
...
EXCEPTION
...
END; -- fin del bloque interior anidado
...
EXCEPTION
...
END
```

Se pueden anidar bloques en las secciones ejecutable y de excepciones, pero no en la declarativa.

3.1.2 Definición de datos compatibles con SQL.

PL/SQL dispone de tipos de datos compatibles con los tipos utilizados para las columnas de las tablas: NUMBER, VARCHAR2, DATE, etcétera, además de otros propios, como BOOLEAN.

Las declaraciones de los datos deben realizarse en la sección de declaraciones:

```
DECLARE
importe NUMBER(8,2);
contador NUMBER(2);
nombre CHAR(20);
nuevo VARCHAR2(15);
BEGIN
...
```

PL/SQL permite declarar una variable del mismo tipo que otra variable o que una columna de una tabla mediante el atributo %TYPE.

Ejemplo: `nombre_act empleados.nombre%TYPE;`

Declara la variable *nombre_act*, del mismo tipo que la columna nombre de la tabla empleados.

También se puede declarar una variable para guardar una fila completa de una tabla mediante el atributo `%ROWTYPE`.

Ejemplo: `mi_fila empleados%ROWTYPE;`

Declara la variable *mi_fila* del mismo tipo que las filas de la tabla *empleados*.

3.1.3 Estructuras de control.

Las estructuras de control de PL/SQL son las habituales de los lenguajes de programación estructurados: IF, CASE, WHILE, FOR y LOOP.

Estructuras de control alternativas		
Alternativa simple <pre>IF <condición> THEN instrucciones; END IF;</pre> Alternativa doble <pre>IF <condición> THEN instrucciones; ELSE instrucciones; END IF;</pre>	Alternativa múltiple (elsif) <pre>IF <condición> THEN instrucciones; ELSIF <condición2> THEN instrucciones; ELSIF <condición3> THEN instrucciones; ... ELSE instrucciones; END IF;</pre>	Alternativa múltiple (case) <pre>CASE [<expresión>] WHEN <test1> THEN <instrucciones1>; WHEN <test2> THEN <instrucciones2>; WHEN <test3> THEN <instrucciones3>; [ELSE <otrasinstrucciones>;] END CASE;</pre>
Estructuras de control repetitivas		
Mientras <pre>WHILE <condición> LOOP Instrucciones; END LOOP;</pre>	Para <pre>FOR <variable> IN <mínimo>..<máximo> LOOP instrucciones; END LOOP;</pre>	Iterar... fin iterar salir si... <pre>LOOP instrucciones; EXIT WHEN <condición>; instrucciones; END LOOP;</pre>

Figura 3.2.1 Formato estructuras de control.

3.1.4 Soporte para órdenes de manipulación de datos

Desde PL/SQL se puede ejecutar cualquier orden de manipulación de datos. Ejemplos:

```
DELETE FROM clientes WHERE nif = v_nif;
```

Borra de la tabla *clientes* la fila correspondiente al cliente cuyo NIF se especifica en la variable *v_nif*.

```
UPDATE productos SET stock_disponible := stock_disponible - unidades_vendidas  
WHERE producto_no = v_producto;
```

Actualiza en la tabla *productos* la columna *stock_disponible*, correspondiente al código de producto especificado en la variable *v_producto*, restándole el valor que se especifica en la variable *unidades_vendidas*.

```
INSERT INTO clientes VALUES (v_num, v_nom, v_loc, ...);
```

Añade a la tabla *clientes* una fila con los valores contenidos en las variables que se especifican.

3.1.5 Soporte para consultas.

PL/SQL permite ejecutar cualquier consulta admitida por la base de datos. Pero cuando se ejecuta la consulta, el resultado no se muestra automáticamente en el terminal del usuario, sino que queda en un área de memoria denominada **cursor** a la que accederemos utilizando variables.

Por ejemplo, para obtener en PL/SQL el número total de empleados de la tabla del mismo nombre no podemos utilizar directamente la instrucción SQL correspondiente pues dará error:

```
SELECT COUNT(*) FROM emple; -- ERROR
```

Utilizaremos una o más variables (declaradas previamente) junto con la cláusula *INTO* para poder acceder a los datos devueltos por nuestra consulta, tal como figura en el siguiente ejemplo:

```
SELECT COUNT(*) INTO v_total_emple FROM emple;
```

Ejecuta la consulta y deposita el resultado en la variable *v_total_emple*, que suponemos declarada previamente. A este tipo de cursores se les denomina **cursores implícitos**, ya que no hay que declararlos. Es el tipo más sencillo, pero

tiene ciertas limitaciones: la consulta deberá devolver una única fila, pues en caso contrario se producirá un error catalogado como *TOO_MANY_ROWS*.

El formato básico es:

SELECT <columna/s> **INTO** <variable/s> **FROM** <tabla> **WHERE**...];

La/s variable/s que siguen a INTO reciben el valor de la consulta. Por tanto, debe haber coincidencia en el tipo con las columnas especificadas en la cláusula SELECT.

3.1.6 Gestión de excepciones.

Las **excepciones** sirven para tratar errores y mensajes de aviso. En Oracle están disponibles excepciones predefinidas correspondientes a algunos de los errores más frecuentes que se producen al trabajar con la base de datos, como, por ejemplo:

- **NO_DATA_FOUND**. Una orden de tipo SELECT INTO no ha devuelto ningún valor.
- **TOO_MANY_ROWS**. Una orden SELECT INTO ha devuelto más de una fila.

Las excepciones se disparan automáticamente al producirse los errores asociados. La sección EXCEPTION es la encargada de gestionar mediante los manejadores (WHEN) las excepciones que puedan producirse durante la ejecución del programa.

Cuando PL/SQL detecta una excepción, automáticamente pasa el control del programa a la sección EXCEPTION del bloque PL/SQL. Allí buscará un manejador (cláusula WHEN) para la excepción producida, o uno genérico (WHEN OTHERS), y realizará el tratamiento establecido. Al finalizar este tratamiento, sale del bloque actual y devuelve el control al programa o a la herramienta que realizó la llamada.

PL/SQL permite que el programador defina sus propias excepciones.

3.1.7 Estructura modular

En esta primera aproximación podemos distinguir los siguientes tipos de programas PL/SQL que se ejecutan en el servidor Oracle:

- Bloques anónimos.
 - 1 No tienen nombre. Se ejecutan en el servidor, pero no se guardan en él.
 - 2 La zona de declaraciones comienza con la palabra DECLARE.
 - 3 Su utilización real es escasa.
- Subprogramas: procedimientos y funciones.
 - 1 Son bloques PL/SQL que tienen un nombre por el que son invocados desde otros programas o herramientas.
 - 2 Se compilan, almacenan y ejecutan en la base de datos Oracle.
 - 3 Tienen una cabecera que incluye el nombre del subprograma (indicando si se trata de un procedimiento o una función), los parámetros y el tipo de valor de retorno en el caso de las funciones.
 - 4 La zona de declaraciones y el bloque o cuerpo del programa comienza con la palabra IS o AS.

- 5 Pueden ser de dos tipos: PROCEDIMIENTOS y FUNCIONES. Su formato genérico es prácticamente el mismo pero las funciones devuelven un valor en algún punto de su código y deben declarar el tipo de valor devuelto en la cabecera.

Procedimiento	Función
<pre> PROCEDURE <nombreprocedimiento> [(<lista de parámetros>)] IS [<declarac objetos locales>;] BEGIN <instrucciones>; [EXCEPTION <excepciones>;] END [<nombreprocedimiento>;] </pre>	<pre> FUNCTION <nombrefunción> [(<lista de parámetros>)] RETURN <tipo valor devuelto > IS [<declaración objetos locales>;] BEGIN <instrucciones>; RETURN <expresión>; [EXCEPTION <excepciones>;] END [<nombrefuncion>;] </pre>

Figura 3.2.2 Estructura de un Procedimiento y una Función en PL/SQL.

Al final de esta unidad podemos ver ejemplos de procedimientos y funciones.

- Disparadores de base de datos.
 - 1 Son programas almacenados en la base de datos que se asocian a un evento.
 - 2 Se ejecutan automáticamente al producirse determinados cambios (inserción, borrado o modificación) en la tabla a la que están asociados o en el sistema.
 - 3 Son muy útiles para controlar los cambios que suceden en la base de datos, para implementar restricciones complejas, etcétera.

3.2 Estructura de un bloque PL/SQL

El bloque de datos es la unidad de almacenamiento de menor tamaño que Oracle puede utilizar o asignar. Un bloque de datos se corresponde con un número específico de bytes de espacio físico en disco. El tamaño del bloque de datos puede configurarse para cada base de datos Oracle en el momento de su creación. Este tamaño del bloque de datos debe ser un múltiplo del tamaño de bloque del sistema operativo (dentro del límite máximo que tenga el sistema) para evitar operaciones innecesarias de E/S. Un bloque de datos tiene la siguiente estructura:

- Cabecera. Contiene información general como la dirección del bloque y el tipo de segmento.

- Directorio de tablas. Contiene información acerca de las tablas que contienen datos dentro de ese bloque de datos.
- Directorio de filas. Contiene información acerca de las filas contenidas en el bloque de datos.
- Datos de las filas. Contiene las filas de datos de las tablas. Una misma fila puede estar dividida entre varios bloques de datos.
- Espacio libre. Asignado para la inserción de nuevas filas y para las actualizaciones de filas existentes que requieren espacio adicional. A partir de Oracle 8i, Oracle puede gestionar de forma automática el espacio libre, aunque existe una opción para hacerla de forma manual.

El siguiente nivel de espacio lógico en la base de datos se denomina extensión. Una extensión es un número específico de bloques de datos contiguos que se asigna para almacenar un tipo específico de información. El nivel situado por encima de las extensiones se denomina segmento. Un segmento es un conjunto de extensiones asignado a una cierta estructura lógica.

Por ejemplo, los datos de cada tabla se almacenan en su propio segmento de datos, mientras que los datos de cada índice se almacenan en su propio segmento de índice. La **Figura 3.3.1** muestra la relación entre bloques de datos, extensiones y segmentos. Oracle asigna dinámicamente el espacio cuando se llenan las extensiones existentes de un segmento. Puesto que las extensiones se asignan según sea necesario, las extensiones de un segmento pueden o no estar contiguas en el disco.

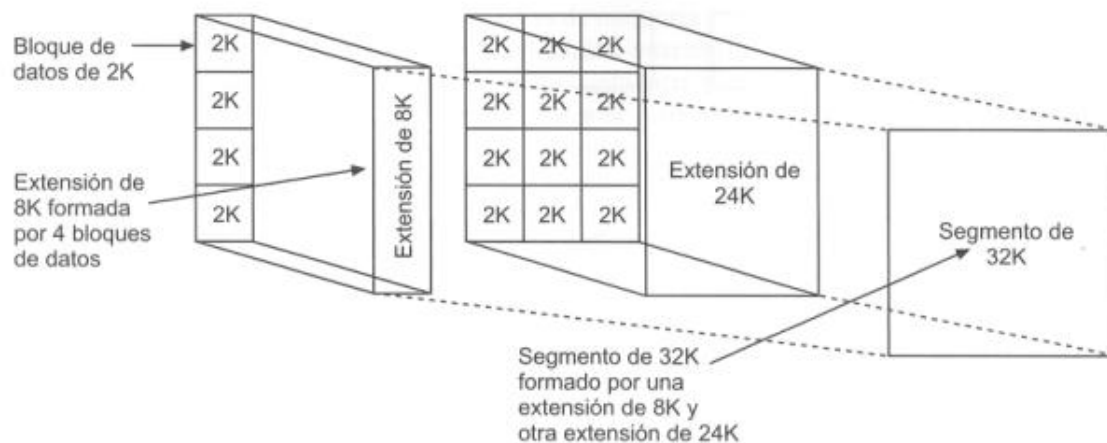


Figura 3.3.1 Relación entre los bloques de datos, las extensiones y los segmentos en Oracle.

[illegible]

- Indicador de comentario de una línea.
- % Indicador de atributo (TYPE, ROWTYPE, FOUND,...).
- : Indicador de variables de transferencia (bind).
- , Separador de ítem de lista.
- ; Terminador de instrucción.
- @ Indicador de enlace de base de datos.

3.3.1 Orden de precedencia en los operadores

El orden de precedencia o prioridad de los operadores determina el orden de evaluación de los operandos de una expresión. Por ejemplo, la siguiente expresión: $12+18/6$, dará como resultado 15, ya que la división se realizará primero. En la siguiente tabla se muestran los operadores disponibles agrupados y ordenados de mayor a menor, por orden de precedencia.

Prioridad	Operador	Operación
1	** , NOT	Exponenciación, negación.
2	* , /	Multiplicación, división.
3	+ , - ,	Suma, resta, concatenación.
4	= , != , < , > , <= , >= , IS NULL , LIKE , BETWEEN , IN	Comparación.
5	AND	Conjunción.
6	OR	Disyunción.

Figura 3.4.2 Orden de precedencia en los operadores.

Debemos tener cuidado con expresiones del tipo:

100 > x > 0 -- ilegal

Ya que primero se ejecutará $100 > x$ y devolverá un valor que será VERDADERO, FALSO o NULL, y ése será el primer término de la siguiente comparación, lo cual causará un error. Para evitarlo, podemos indicar la expresión de la siguiente forma:

(100 > x) AND (x > 0)

Aunque ésta es la prioridad establecida por defecto, podemos cambiarla utilizando paréntesis. Por ejemplo, en la expresión anterior, si queremos que la suma se haga antes que la división, lo indicaremos con la expresión: $(12+18)/6$. El resultado será 5.

Los operadores que se encuentran en el mismo grupo tienen la misma precedencia. En estos casos no se garantiza el orden de evaluación. Si queremos que se evalúen en algún orden concreto, deberemos utilizar paréntesis.

3.3.2 Evaluación en cortocircuito

PL/SQL da por concluida la evaluación de una expresión lógica cuando el resultado ha quedado determinado por la evaluación de una parte de ella. Esto ocurre cuando en una operación OR el primer operando es verdadero, o en una operación AND el primer operando es falso. De esta forma, los programas se ejecutan más rápidamente.

La evaluación en cortocircuito se puede aprovechar para evitar algunos problemas. Por ejemplo, suponemos que x e y son variables. Si queremos comprobar que, y / x es mayor que 1, podemos evitar la posibilidad de un error ORA-01476 (intento de división por cero) aprovechando la evaluación en cortocircuito:

```
IF (x != 0) AND (y / x > 1) THEN ...
```

3.4 Estructuras de control en PL/SQL

PL/SQL dispone de estructuras para controlar el flujo de ejecución de los programas.

ESTRUCTURAS ALTERNATIVAS:	
Alternativa simple	
<pre>IF <condicion> THEN instrucciones;END IF;</pre>	Si la condición se cumple, se ejecutan las instrucciones que siguen a la cláusula THEN.
Alternativa doble	
<pre>IF <condicion> THEN instrucciones1;ELSE instrucciones2;END IF;</pre>	Si la condición se cumple, se ejecutan las instrucciones que siguen a la cláusula THEN. En caso contrario, se ejecutarán las instrucciones que siguen a la cláusula ELSE.
Alternativa múltiple con ELSIF	
<pre>IF <condicion1> THEN instrucciones1; ELSIF <condicion2> THEN instrucciones2; ELSIF <condicion3> THEN instrucciones3; ... [ELSE instruccionesotras;] END IF;</pre>	Evalúa, comenzando desde el principio, cada condición, hasta encontrar alguna condición que se cumpla, en cuyo caso ejecutará las instrucciones que siguen a la cláusula THEN correspondiente. La cláusula ELSE es opcional; en caso de que se utilice, se ejecutará cuando no se ha cumplido ninguna de las condiciones anteriores.

Figura 3.5.1 Tipos de estructuras alternativas.

Alternativa múltiple con CASE de comprobación	
<pre> CASE expresión WHEN <valorcomprobac1> THEN instrucciones1; WHEN <valorcomprobac2> THEN instrucciones2; WHEN <valorcomprobac3> THEN instrucciones3; ... [ELSE instruccionesotras;] END CASE; </pre>	Calcula el resultado de la expresión que sigue a la cláusula CASE. A continuación, comprueba si el valor obtenido coincide con alguno de los valores especificados detrás de las cláusulas WHEN, en cuyo caso ejecutará la instrucción o instrucciones correspondientes. La cláusula ELSE es opcional; se ejecutará en caso de que no se encuentre un valor coincidente en las cláusulas WHEN precedentes.
Alternativa múltiple con CASE de búsqueda	
<pre> CASE WHEN <condicion1> THEN instrucciones1; WHEN <condicion2> THEN instrucciones2; WHEN <condicion3> THEN instrucciones3; ... [ELSE instruccionesotras;] END CASE; </pre>	Evalúa, comenzando desde el principio, cada condición, hasta encontrar alguna condición que se cumpla, en cuyo caso ejecutará las instrucciones que siguen a la cláusula THEN correspondiente. La cláusula ELSE es opcional; en caso de que se utilice, se ejecuta cuando no se ha cumplido ninguna de las condiciones anteriores.

Figura 3.5.2 Tipos de estructuras alternativas.

En ocasiones, se puede usar NULL como una instrucción que no hace nada por motivos de claridad o facilidad de codificación:

```

CASE val
WHEN 1 THEN
INSERT INTO TEMP VALUES 'UNO';
WHEN 2 THEN
INSERT INTO TEMP VALUES 'DOS';
WHEN 0 THEN
NULL; -- No hace nada.
ELSE THEN
INSERT INTO TEMP VALUES 'ERROR'
END CASE;

```

En un bucle WHILE, si la condición no se cumple al comienzo, no se ejecutará ni una sola línea pues la comprobación se hace antes de entrar en el bucle y posteriormente, una vez finalizadas todas las instrucciones que incluye. LOOP, sin embargo, siempre entrará en el bucle, ejecutará las primeras instrucciones y saldrá del bucle cuando se cumpla la condición.

ESTRUCTURAS REPETITIVAS	
Iterar	
<pre> LOOP Instrucciones; EXIT [WHEN <condición>]; instrucciones; END LOOP; </pre>	Se trata de un bucle que se repetirá indefinidamente hasta que encuentre una instrucción EXIT sin condición o hasta que se cumpla la condición asociada a la cláusula EXIT WHEN. Es una condición de salida.
Mientras	
<pre> WHILE <condicion> LOOP instrucciones; END LOOP; </pre>	Se evalúa la condición y, si se cumple, se ejecutarán las instrucciones del bucle. El bucle se seguirá ejecutando mientras se cumpla la condición. Es una condición de continuación.

Figura 3.5.3 Tipos de estructuras repetitivas.

EXIT es una instrucción en sí misma (por eso lleva punto y coma al final) y puede ser utilizada con o sin la cláusula WHEN.

```

LOOP
instrucciones;
IF <condición> THEN
EXIT;
END IF;
instrucciones;
END LOOP;

```

PL/SQL permite la utilización de EXIT incluso en otros bucles y estructuras. Esta práctica es totalmente desaconsejable.

Para
<pre> FOR <variablecontrol> IN <valorInicio>..<valorFinal> LOOP instrucciones; END LOOP; </pre>

[illegible]

Procedimiento PL/SQL terminado con éxito.

Podemos indicar los valores mínimo y máximo mediante expresiones:

```
DECLARE
Num1 INTEGER;
Num2 INTEGER;
...
BEGIN
...
FOR x IN Num1..Num2 LOOP
...
END LOOP;
...
END;
```

Respecto a la variable de control, hay que tener en cuenta que:

- No hay que declararla.
- Es local al bucle y no es accesible desde el exterior del bucle, ni siquiera en el mismo bloque.
- Se puede usar dentro del bucle en una expresión, pero no se le pueden asignar valores.

En el siguiente ejemplo, definimos una variable e intentamos usarla como variable de control. Aun en ese caso la estructura creará la suya propia como local, quedando la nuestra como global en el bucle:

```
<<ppal>>
DECLARE
i INTEGER;
BEGIN
...
FOR i IN 1..10 LOOP
...
/* cualquier referencia a i será entendida como a la
variable local al bucle. Si quisiéramos referirnos a
la otra lo debemos hacer como ppali */
END LOOP;
...
/*La variable local del bucle ya no existe aquí */
END ppal;
```